# Step 2a — Dataset: le fonti esistenti, rilette per il compito binario

## Obiettivo

Riusare le coppie (documento, claim, etichetta) già prodotte da `data_prep/build_pairs.py` (../fine_tuning/02a_dataset_fonti_esistenti.md) e decidere **quanto pesa ciascuna fonte** quando l'unico obiettivo è il verdetto binario. In più: costruire un **sottoinsieme difficile di ANLI** come fa MiniCheck, e aggiungere le fonti multi-hop da cui FactCG genera CG2C-MHQA.

## 1. Cosa cambia con il compito binario

| Aspetto | Piano generale | Qui |
|---|---|---|
| `orig_label` a 3 classi | serviva per i target delle domande 2 e 3 | serve solo per analisi (errori su "contraddetto" vs "non stabilito") e per bilanciare i due tipi di negativo |
| Formato | stato JSON di un decision model | invariato: il prompt resta quello nativo, con una sola domanda |
| Etichetta | 1 = supportato in ogni parte; 0 = contraddetto **o** non stabilito | identica (convenzione LLM-AggreFact) |

Regola di qualità per i negativi: per un fact checker binario i negativi **"non stabilito"** (manca un pezzo di evidenza) sono i più utili e i più rari nei dataset pubblici. Il mix deve averne almeno ~40% sul totale dei negativi; il resto contraddetti.

## 2. Le fonti e il loro ruolo

| Fonte | Coppie train disponibili | Ruolo nel compito binario | Stadio (02e) |
|---|---|---|---|
| **MiniCheck C2D** | ~7k | claim multi-fatto, negativi per evidenza tolta: è il dato costruito per questo compito | 1 |
| **MiniCheck D2C** | ~7k | stessa logica su notizie reali | 2 |
| **ANLI R1–R3** | 162.865 | inferenza difficile; **solo il sottoinsieme difficile** (§3) | 1 |
| SummEdits | 6.348 | riassunti con errori sottili, 10 domini: vicino alle fonti deboli di AggreFact (CNN, XSum, MediaSum) | 2 |
| HoVer | 17.000 | multi-hop su 2–4 pagine | 1 |
| FEVER | 144.411 | single-hop, molti NEI; scorciatoie note | 1 (quota bassa) |
| VitaminC | 370.653 | contrasti su numeri e date (coppie minime) | 1 |

| Fonte | Coppie train disponibili | Ruolo nel compito binario | Stadio (02e) |
|---|---|---|---|
| AVeriTeC | 2.873 | claim reali, evidenza in forma domanda–risposta | 2 (piccola) |
| HotpotQA, MuSiQue (**nuove**) | ~90k / ~20k domande | non si usano direttamente: sono il materiale di partenza di CG2C-MHQA (02b, §3) | — |
| FactKG | 81.342 | solo dopo la trasformazione in prosa, ed è un esperimento (D-FKG) | 1 |
| SciFact | 919 | **test_ood**, non in training | — |
| WiCE, RAGTruth | — | esclusi (contaminazione con AggreFact) | — |

Per HotpotQA e MuSiQue va aggiunto un builder in `build_pairs.py` (HF: `hotpotqa/hotpot_qa` config `distractor`, `bdsaglam/musique` o equivalente) e la decontaminazione: le loro pagine sono Wikipedia come FEVER e HoVer, il rischio verso AggreFact è basso ma va misurato.

## 3. ANLI difficile (come MiniCheck)

MiniCheck non usa tutto ANLI: usa **21k esempi su cui i loro modelli di entailment sbagliavano**. Il senso: gli esempi facili non insegnano nulla al modello e ne spostano la calibrazione.

Con il nostro modello di partenza:

1. Inferenza di KnowLine-4B (una domanda) su tutto ANLI train (162.865 righe, premesse corte: ~130 token p95), con `benchmark.py` sul file esportato come dataset locale.
2. Per ogni riga, `p` = P(supportato). Errore "sicuro" se label 1 e `p < 0,3`, oppure label 0 e `p > 0,7`. Errore "incerto" se `0,3 ≤ p ≤ 0,7`.
3. Si tengono tutti gli errori sicuri e incerti, più un 20% di esempi giusti scelti a caso (per non insegnare che "ANLI = sempre il contrario di quello che pensi").
4. Bilanciamento 50/50 e tetto a ~20k.

Attenzioni:

- la selezione si fa **solo sul train** di ANLI;
- se si sceglie con KnowLine-4B e si addestra un altro modello, gli esempi difficili sono un po' meno adatti a quel modello: va bene (MiniCheck usava modelli diversi da quelli addestrati); in alternativa una selezione per modello;
- una parte degli errori "sicuri" sono **etichette sbagliate** di ANLI: con la CE spingono forte. Se la quota di errori sicuri supera ~30%, usare Brier o label smoothing su queste righe (04).

Esempio illustrativo (costruito nello stile di ANLI, non una riga reale; label entailment → 1):

> Premessa: "… The company reported a net loss of $2 million in the third quarter, compared with a profit a year earlier." Claim: "The company was profitable in the third quarter of the previous year."

Un modello che guarda la somiglianza lessicale tende a dire "no" (la premessa parla di perdita); ma "compared with a profit a year earlier" stabilisce proprio il claim. Errori così sono il materiale utile.

## 4. Le scorciatoie da tenere sotto controllo

| Fonte | Scorciatoia | Controllo |
|---|---|---|
| FEVER | negazioni nel claim (“not”, “never”) → falso | test “solo claim”; esperimento senza negazioni esplicite |
| FEVER (NEI) | documento scelto per titolo → negativi facili | quota bassa (≤ 10% del mix) |
| VitaminC | evidenza di 1–2 frasi | quota ≤ 15%; documenti corti non rappresentano l’uso reale |
| ANLI | premesse corte | combinarlo con fonti lunghe |
| SummEdits | modifiche al limite dell’opinabile | Brier o smoothing se pesa |

## 5. Comandi

```
conda run -n factcheck python data_prep/build_pairs.py all            # già fatto
conda run -n factcheck python data_prep/build_pairs.py hotpotqa musique  # da aggiungere
conda run -n factcheck python finetune/mine_hard.py --source anli --model knowline-4b   # da scrivere (§3)
```

## Criteri per passare allo step successivo

- `anli_hard.jsonl` prodotto, con la distribuzione di `p` e la quota di errori sicuri registrate.
- Builder HotpotQA/MuSiQue scritti e decontaminati (se si fa CG2C-MHQA).
- 20 righe per fonte lette a mano, con attenzione ai negativi “non stabilito”.